Here is a small Azure CLI script along with the accompanying reflective report.

bash

```bash
#!/bin/bash

# Small Azure configuration script
# Creates a resource group and a storage account with basic settings

RESOURCE_GROUP="DevTeamProject-Storage-RG"
LOCATION="uksouth"
STORAGE_ACCOUNT="devteamstorage2026"
CONTAINER_NAME="projectfiles"

echo "Creating resource group: $RESOURCE_GROUP"
az group create --name "$RESOURCE_GROUP" --location "$LOCATION"

echo "Creating storage account: $STORAGE_ACCOUNT"
az storage account create \
  --name "$STORAGE_ACCOUNT" \
  --resource-group "$RESOURCE_GROUP" \
  --location "$LOCATION" \
  --sku Standard_LRS \
  --kind StorageV2

echo "Retrieving storage account key"
ACCOUNT_KEY=$(az storage account keys list \
  --resource-group "$RESOURCE_GROUP" \
  --account-name "$STORAGE_ACCOUNT" \
  --query "[0].value" -o tsv)

echo "Creating blob container: $CONTAINER_NAME"
az storage container create \
  --name "$CONTAINER_NAME" \
  --account-name "$STORAGE_ACCOUNT" \
  --account-key "$ACCOUNT_KEY" \
  --public-access off

echo "Storage account and container created successfully."
```

500 word report: experience and challenges of building a small Azure configuration

For this task, I used the Azure CLI to write a small script that provisions a resource group, a storage account and a private blob container. I chose Azure CLI rather than the Azure Portal because it aligns with the automation focus of the module and reflects the same approach used in the earlier Bash provisioning exercise, where infrastructure is defined and executed through code rather than manual configuration.

The overall experience of building the script was reasonably smooth once the correct sequence of commands had been identified. Azure CLI follows a fairly logical structure, where a resource group is created first, followed by the specific resource that needs to sit within it. Wiggins (2017) notes that treating infrastructure

and configuration consistently across environments reduces the likelihood of errors, and writing this script reinforced that idea directly, since every value needed for later steps, such as the storage account name, had to be defined clearly and referenced consistently throughout.

One of the main challenges encountered was around naming constraints. Azure storage account names must be globally unique across the entire platform, lowercase, and limited to a set number of characters, which is a stricter requirement than resource group names. Several attempts had to be made before finding a name that satisfied these constraints, which highlighted a gap between how straightforward the task initially appears and the practical constraints imposed by the platform itself. This is a good example of a challenge that is easy to overlook until the script is actually executed against a live environment rather than only reviewed in theory.

A second challenge related to retrieving the storage account key securely and passing it into the subsequent command. Since the key is generated dynamically after the storage account is created, it had to be captured using a variable within the script rather than hard coded, which reflects the principle raised by Wiggins (2017) regarding strict separation between configuration values and application logic. This step required a closer reading of the Azure CLI documentation to identify the correct query syntax for extracting the key from the JSON output returned by the initial command.

More broadly, this exercise reinforced how much of cloud configuration work involves smaller, easily missed details rather than large architectural decisions. Hamzaoui et al. (2024) discuss how container based and cloud native approaches introduce additional complexity around configuration and resource management compared to traditional deployment models, and although this task did not involve containers directly, the same underlying tension between simplicity of intent and complexity of execution was clearly present when working through naming rules, permissions and key management.

Overall, the task was a useful, low risk way to practice writing cloud configuration as code, and it highlighted the importance of testing scripts against a live environment rather than assuming correctness from the command syntax alone. It has also given me a clearer appreciation of why automated provisioning, despite its initial setup overhead, ultimately reduces the risk of configuration errors compared to manual processes.

References

Hamzaoui, I. et al. (2024) 'A topical review on container based cloud revolution: multi directional challenges, and future trends', *SN Computer Science*, 5(1), pp. 1 to 20.

Wiggins, A. (2017) *The twelve factor app*. Available at: https://12factor.net (Accessed: 23 August 2026).